PROGRAMMING IN HASKELL

Topic 12 – Monads

Recap on functor

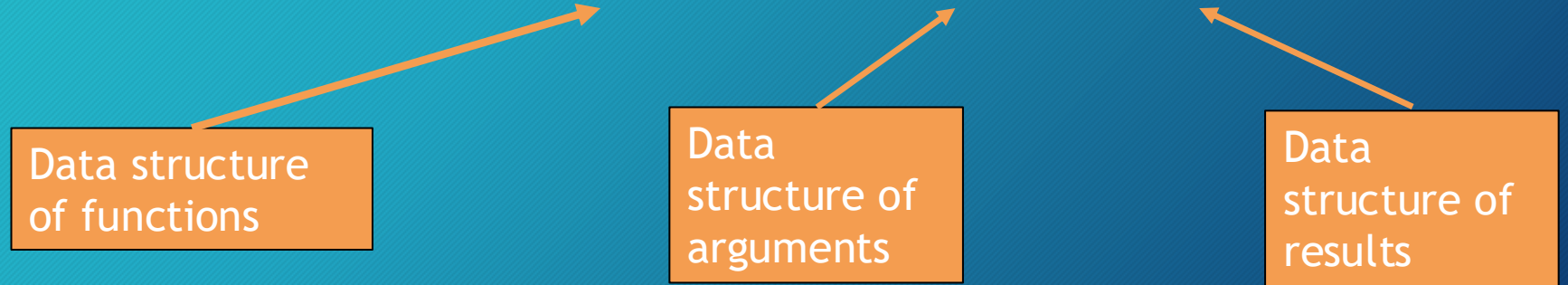

```
class Functor f where
   fmap :: (a-> b) -> f a -> f b
```

```
➢ fmap (+2) [1,2,3]
[3,4,5]
➢ fmap (+2) (Just 1)
Just 3
➢ fmap (+2) Nothing
Nothing
```

Recap on Applicative

- Generalise the idea of Functors to functions with more than one argument or parameter
- This gives rise to an Applicative Functor

```
class Functor f => Applicative where
  pure :: a -> f a
  ( <*> ) :: f (a-> b) -> f a -> f b
```



Examples


```
➢ pure (+) <*> Just 1 <*> Just 2
Just 3
➢ pure (+) <*> Nothing <*> Just 2
Nothing
```

Applicative handles Errors (Nothing) correctly.

Introduce Monads – Example a simple evaluator


```
data Expr = Val Int | Div Expr Expr
e.g.
e:: Expr
e = Div (Val 6) (Val 3)
```

```
eval :: Expr -> Maybe Int

eval (Val n)       = n
eval (Div  x y )  = eval x `div` eval y
```

Problem – how do we deal with division by zero (this would crash)

Need to refine program to deal with this

- How to capture failure

```
safeDiv :: Int -> Int -> Maybe Int

safeDiv _ 0   = Nothing
safeDiv n m   = Just (n `div` n)
```

Now, rewrite eval to use safeDiv


```
eval:: Expr -> Maybe Int

eval (Val n) = Just n
eval (Div x y) = case eval x of
                   Nothing -> Nothing
                   Just n -> case eval y of
                        Nothing -> Nothing
                        Just m -> safeDiv n m
```

Can we simplify?

Problem with Applicative


```
eval:: Expr -> Maybe Int

eval (Val n) = pure n
eval (Div x y) = pure safeDiv <*> eval x <*> eval y
```

Applicative needs Int -> Int -> Int

But we have Int -> Int -> Maybe Int

Maybe Int

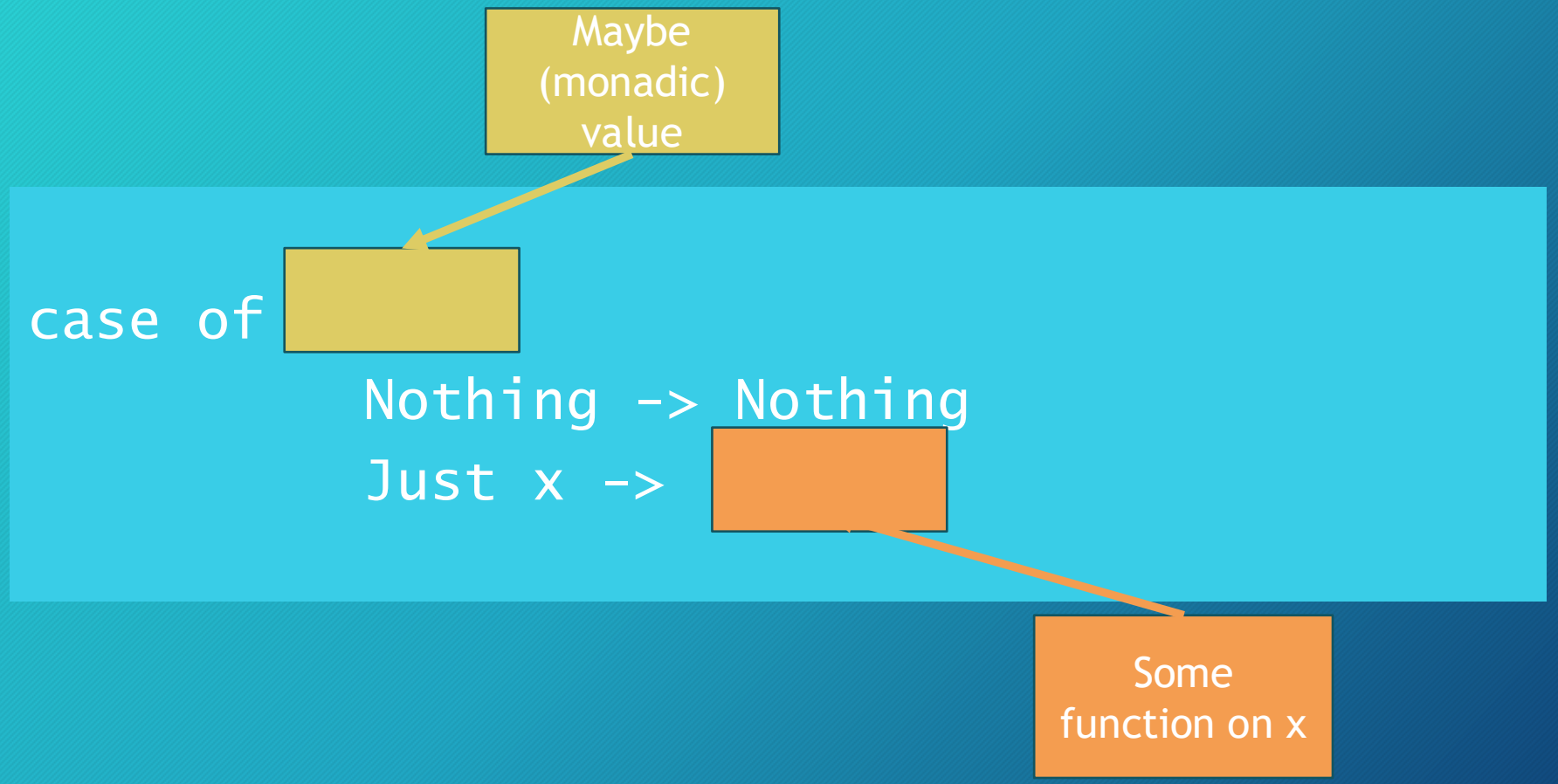
Maybe Int

Monads – observe a common pattern and abstracting it out


```
eval:: Expr -> Maybe Int

eval (Val n) = Just n
eval (Div x y) = case eval x of
                   Nothing -> Nothing
                   Just n -> case eval y of
                               Nothing -> Nothing
                               Just m -> safeDiv n m
```

See pattern


Maybe (monadic) value
case of
Nothing -> Nothing
Just x ->
Some function on x

See pattern


```
case mx of
          Nothing -> Nothing
          Just x ->  f x
```

We want to turn this into a definition.

See pattern


```
case mx of
     Nothing -> Nothing
     Just x ->  f x
```

We want to turn this into a definition.

```
mx >>= f = case mx of
           Nothing -> Nothing
           Just x ->  f x
```

“into” “bind”

```
(>>=) :: Maybe a -> (a-> Maybe b) -> Maybe b
```

>>=

New evaluation function using >>=


```
eval :: Expr -> Maybe Int
eval (Val n)   = Just n
eval (Div x y) = eval x >>= (\n ->
                 eval y >>= (\m ->
                 safeDiv n m)    )
```

General pattern


```
m1 >>= \x1 ->
m2 >>= \x2 ->
m3 >>= \x3 ->
:
:
mn >>= \xn ->
f x1 x2    .. xn
```

Syntactic sugar

```
"This is what we
use"
do x1  <- m1
   x2  <- m2
   x3  <- m3

   xn  <- mn
   f x1 x2 x3 .. xn
```

New evaluation function using do (final version)


```
eval :: Expr -> Maybe Int
eval (Val n)   = Just n
eval (Div x y) = do
                   n <- eval x
                   m <- eval y
                   safeDiv n m
```

So now we are ready for Monads

Now that we have bind (>>=) we now can define this kind of applicative that has bind.

```
class Applicative m => Monad m where
  (>>= ) :: m a -> (a -> m b) -> m b
  return :: a -> m a
  return = pure
```

Example - Maybe – already have Functor, Applicative defs


```
data myMaybe a = Just a | Nothing deriving (Eq, Show)

instance Functor Maybe where
  fmap f (Just x) = Just (f x)
  fmap _ _        = Nothing
instance Applicative Maybe where
   pure = Just
   Nothing <*> _ = Nothing
   (Just f ) <*> something = fmap f something
```

Example - Maybe – already have Functor, Applicative defs


```
data myMaybe a = Just a | Nothing deriving (Eq, Show)

instance Functor Maybe where
  fmap f (Just x) = Just (f x)
  fmap _ _        = Nothing
instance Applicative Maybe where
   pure = Just
   Nothing <*> _ = Nothing
   (Just f ) <*> something = fmap f something
```

Example - Maybe – already have Functor, Applicative defs


```
data myMaybe a = Just a | Nothing deriving (Eq, Show)

instance Functor Maybe where
  fmap f (Just x) = Just (f x)
  fmap _ _        = Nothing
instance Applicative Maybe where
  pure = Just
  Nothing <*> _ = Nothing
  (Just f ) <*> something. = fmap f something
```

```
instance Monad Maybe where
   -- (>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b)
   Nothing >>= f  = Nothing
   Just x  >>= f = Just (f x)
```

Example - Lists


```
instance Monad [] where
   -- (>>=) :: [a] -> (a -> [b] ) -> [b]
      Nothing >>= f  = Nothing
      xs >>= f  = concat  map f xs
```

Note the concat

or

```
instance Monad [] where
   -- (>>=) :: [a] -> (a -> [b] ) -> [b]
      Nothing >>= f  = Nothing
      xs >>= f  = [y| x <- xs, y <- f x]
```

Example - using List monads


```
➢ pairs [1,2] [3,4]   -- cartesian product
[(1,3), 1,4), (2,3), 2,4)]
```

```
pairs :: [a] -> [b] -> [(a,b)]
pairs xs ys = do x <- xs
                 y <- ys
                 return (x,y)
```

Monads

A monad is defined by 3 things:

- A type constructor m
- A function return
- An operator (>>=) – read as "bind"

Originally, these were introduced for IO, but they can be much more powerful.

```
return :: a -> m a
(>>=) :: m a -> (a -> m b) -> m b
```

The function and operator are methods of the Monad typeclass and have types themselves, which are required to obey 3 laws (more later):

Monad laws

Monads are required to obey 3 laws:

```
m  >>= return     = m
return x >>= f    = f x
(m >>= f) >>= g   = m >>= (\x -> f x >>= g) -- associativity
```

Monads originally come from a branch of mathematics called Category Theory.

(Fortunately, it is entirely unnecessary to understand category theory in order to understand and use monads in Haskell.)

Monadic Programming

Having designed a Monad (i.e. 'I need a Monad with this shape'), we define an instance of a Monad.

This involves essentially writing what

1. return and
2. >>= mean in this Monad

(this assumes you have written instances for Functor and Applicative)

Examples of monadic programming – 1 - fmap


```
--- fmap using IO
fmap :: (a -> b) ->  IO a -> IO b
```

Given

- a function **f :: a -> b** and
- an IO interaction **m :: IO a**,

run the interaction,

- then apply **f** to the result, and
- wrap it back in IO.

fmap


```
fmap :: (a -> b) ->  IO a -> IO b
```

Given

- a function **f :: a -> b** and
- an IO interaction **m :: IO a**,

We want to:

- run the interaction,
- then apply f to the result, and
- wrap it back in IO.

```
fmap f m =
    do
        x <- m
        return(f x)
```

Examples of monadic programming- 2 - using recursion - repeat


```
repeat :: IO Bool -> IO () -> IO ()
So that
   repeat test oper
```
keeps running oper until test returns True.

Recursion

Base Case

```
do res <- test
   if res
     then return ()
-- done, stop here
```

Recursive case

```
else do m        -- run the operation
   repeat test oper  -- recurse
```

Examples of monadic programming– 2 – using recursion - repeat

Full solution:

```
repeat :: IO Bool -> IO () -> IO ()
repeat test oper
  = do res <- test          ----------   run test, bind result to res
       if res               ----------   if True → done (base case)
         then return ()     ----------   if False → run oper then recurse
         else do oper
                 repeat test m
```

Some more useful monadic functions


```
sequence :: Monad m => [m a] -> m [a]
sequence [] = return []
sequence (m:ms) = do
        x <- m
        xs <- sequence ms
        return (x:xs)
```

Example runs:

- sequence [Just 1, Just 2, Just 3] -- Just [1, 2, 3]
- sequence [Just 1, Nothing, Just 3] -- Nothing

Some more useful monadic functions


```
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
mapM f xs = sequence (map f xs)
```

`map f xs` produces a list of monadic actions, and `sequence` runs them left to right, threading the monad through and collecting results.

Example runs:

- mapM safeHead [[1,2], [3,4], [5,6]] -- Just [1,3,5]
- mapM safeHead [[1,2], [], [5,6]] -- Nothing (one failure kills the whole thing)


ANY
QUESTIONS?